

Log Anomaly Detection via Transformers Pre-Trained on Massive Unlabeled Data

Senming Yan , *Graduate Student Member, IEEE*, Lei Shi , *Graduate Student Member, IEEE*,
Jing Ren , *Member, IEEE*, Wei Wang , *Senior Member, IEEE*, Limin Sun , and Wei Zhang , *Fellow, IEEE*

Abstract—Cyber attacks pose serious threats to computer systems. Automatically detecting anomalous patterns in system logs is critical for identifying and mitigating security risks. However, as log data grows increasingly complex and labeled logs remain scarce, existing detection methods face significant challenges. To address these issues, we introduce the pre-training and fine-tuning paradigm for log analysis and propose a hybrid pipeline tailored for accurate and low-cost log anomaly detection. Specifically, we employ a masked log reconstruction strategy to pre-train a Transformer encoder-based foundation model by leveraging the sequential dependencies in unlabeled logs. The model is then fine-tuned on an event prediction task to derive the anomaly detector. To reduce computational and storage overhead, we further design a knowledge distillation method tailored for compressing log anomaly detectors. Beyond fitting the detector's outputs, our method also exploits its internal representations to transfer richer knowledge. Experiments on the HDFS, BGL, and Thunderbird public datasets demonstrate that our framework outperforms state-of-the-art baselines in multiple metrics. Empirical evaluation on a reconstructed HDFS dataset confirms that it can adapt to real-world scenarios where labeled data is scarce. Moreover, through our knowledge distillation approach, the lightweight detectors achieve outstanding performance with substantially lower overhead, while maintaining robustness in real-world scenarios.

Index Terms—Cyber security, log analysis, log anomaly detection, pre-trained model.

Received 22 October 2025; revised 18 December 2025; accepted 9 January 2026. Date of publication 15 January 2026; date of current version 27 January 2026. This work was supported in part by the Major Key Project of Pengcheng Laboratory under Grant PCL2024A03, in part by the National Key Research and Development Program of China under Grant 2020YFA0711400, in part by the National Natural Science Foundation of China under Grant U20A20156 and Grant 92467201, in part by the Key Research Project of Chinese Academy of Sciences under Grant RCJJ-145-24-17, and in part by Beijing Natural Science Foundation under Grant L234033. Recommended for acceptance by Dr. Yuan Wu. (Corresponding author: Wei Wang.)

Senming Yan and Limin Sun are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100045, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China (e-mail: yansenming@iie.ac.cn; sunlimin@iie.ac.cn).

Lei Shi is with the School of Integrated Circuits (School of Electronic Information and Electrical Engineering), Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: shi.lei@sjtu.edu.cn).

Jing Ren is with the Department of Strategic and Advanced Interdisciplinary Research, Peng Cheng Laboratory, Shenzhen 518066, China, and also with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: renjing@uestc.edu.cn).

Wei Wang is with the Department of Strategic and Advanced Interdisciplinary Research, Peng Cheng Laboratory, Shenzhen 518066, China (e-mail: wei_wang@ieee.org).

Wei Zhang is with the School of Electrical Engineering and Telecommunications, The University of New South Wales, Sydney NSW 2033, Australia (e-mail: w.zhang@unsw.edu.au).

Digital Object Identifier 10.1109/TNSE.2026.3654089

I. INTRODUCTION

A. Background and Motivations

WITH the rapid advancement of modern information technologies such as cloud computing, artificial intelligence, and the Internet of Things (IoT), cyberattacks have become a major threat to cyberspace [2], [3], [4], [5] and are increasingly extending into the physical world [6]. Exploiting networks as a medium, attackers adopt sophisticated and diverse techniques to compromise the availability, integrity, and confidentiality of computer systems [7]. To identify and mitigate such cyber threats, a practical approach is to analyze system operation logs to detect anomalies and implement corresponding countermeasures.

Modern computer systems generate logs to record their critical runtime information [8]. The ever-increasing complexity of computer systems and the exponentially growing amount of log data render manually analyzing logs infeasible [9]. Consequently, automatic log analysis and anomaly detection have been developed as effective technologies to monitor the security status and therefore safeguard computer systems [10], [11], [12], [13].

Log anomaly detection refers to identifying atypical behaviors that fall outside the expected patterns of normal system operations. The anomalies in logs indicate the presence of system failures and even malicious attacks. System operators leverage log anomaly detection methods to identify anomalies before further taking mitigation measures. Log anomaly detection methods utilize structured log data as inputs, which are typically generated by parsing raw logs. The objective of log anomaly detection is to accurately classify input logs as either normal or abnormal. Its core methodologies focus on analyzing the discernible patterns within logs and distinguishing normal log patterns from abnormal ones.

Numerous log anomaly detection methods are proposed in existing works. Some researchers propose to extract quantitative features from log sequences and leverage classic machine learning (ML)-based algorithms to detect anomalous patterns [14], [15]. Different from classic ML-based methods, since logs are essentially structured time-series event sequences, they can be processed as natural language. Therefore, sequential deep learning (DL)-based models, even Large Language Models (LLMs) are leveraged for analyzing logs and detecting anomalies [12], [16], [17].

Despite achieving high detection performance, existing methods still face some significant challenges, making them difficult

to apply in real-world scenarios: (i) as modern computer systems evolve, the types of system logs become more diverse, and the volume of log data is exploding, leading to the demand for developing more powerful log anomaly detectors; (ii) existing methods commonly train anomaly detectors on labeled logs, which are scarce in real-world scenarios, as the amount of log data is extremely large and manual labeling requires heavy labor work; (iii) most existing works only consider the performance metrics of anomaly detectors but ignore the computational and storage overhead issues, limiting their practical deployment.

B. Our Contributions

Since logs are essentially time series data, we design a Transformer encoder-based model to acquire both learning capabilities and computing efficiency. We propose a hybrid training method following the pre-training and fine-tuning paradigm as in Large Language Models [18]. Specifically, to exploit the information within massive unlabeled logs, i.e., the volume of unlabeled log data is significantly larger than manually labeled ones, we pre-train a foundation model to learn the sequential features in logs with an input-masking-inspired self-supervised training task, i.e., masked log reconstruction. Then, we fine-tune it for the event prediction task to discern abnormal patterns and gain advanced detection capabilities for complex log data. To reduce the computational and storage overhead of our log anomaly detector, we propose a knowledge distillation method to compress the fine-tuned model while preserving its detection performance. We also implement a framework for online anomaly detection, including data processing, model training, model compression, and anomaly detection modules. We evaluate our log anomaly detection framework on three widely used public datasets, including HDFS, BGL, and Thunderbird. The evaluation results demonstrate the advantages of our framework.

To summarize, in this work, we have mainly made the following contributions:

- We propose the masked log reconstruction approach to pre-train a Transformer-based foundation model and fine-tune it for the event prediction task to train the detector. Inspired by input masking-based pre-training objectives, our proposed training approach effectively exploits the bidirectional sequential information of log data.
- We propose a hybrid pipeline tailored for log anomaly detection. Different from existing works, our framework follows the pre-training and fine-tuning paradigm, which exploits unlabeled data and reduces the demand for manual labeling.
- We propose a multi-level knowledge distillation approach for log anomaly detection, which leverages both internal and external information from Transformer-based anomaly detectors. The distilled detector has high computational and storage efficiency while maintaining outstanding detection capabilities.
- We evaluate our framework on three public datasets, and our framework outperforms the existing state-of-the-art methods. Also, we simulate real-world scenarios where

unlabeled logs are hard to acquire. Our framework is the one that is still effective in such scenarios.

C. Paper Structure

The remainder of this paper is organized as follows. We introduce related works and present essential background knowledge regarding system log analysis, generative pre-trained transformers, and knowledge distillation in Section II. Then, we give an overall introduction to our hybrid log anomaly detection framework in Section III. After that, in Section IV, we describe our pre-training, fine-tuning, knowledge distillation, and anomaly detection approaches in detail. We introduce the experiment settings in Section V. We conduct several experiments and analyze the results in Section VI. Finally, conclusions and some future work directions are presented in Section VII.

II. RELATED WORK

In this section, we review the existing research works on system log analysis, log anomaly detection, generative pre-trained Transformers, and knowledge distillation, to provide sufficient background and identify the main research gaps.

A. System Log Analysis

Modern computer systems generate logs to record their critical runtime events. By analyzing logs, system operators are capable of monitoring the operational status of computer systems and taking measures when anomalies occur. Therefore, as the requirements for the availability of computer systems increase, it is critical to effectively analyze system logs to maintain high reliability [19], [20].

System logs are structured text printed by logging software, i.e., loggers. Generally, loggers generate log entries by populating predefined templates with parameters that record the critical operating status. In order to extract effective information for system administrators from massive log data, researchers have made significant progress in a wide range of fields such as log parsing, log anomaly detection, and system failure detection [8], [19].

Since logs are essentially semi-structured data generated by logging software, it is significant for automatic analysis tools to extract key information from raw logs and generate structured data, which is the purpose of log parsing techniques. An example of log parsing is illustrated in Fig. 1. A raw log message generated by the Hadoop Distributed File System (HDFS) is processed by the log parser. The log message is a natural language-like string containing information about a system event, including time, level, operations, etc. Log parsers aim to extract this information into a structured format. However, the contents of the log messages are difficult to extract, since their formats are customized by developers. To this end, log parsing methods are proposed to automatically identify the format of log messages, i.e., log templates, to provide structured data for the subsequent log analysis process. As a necessary step in automatic log analysis, researchers have proposed a variety of log parsing methods [21], [22], [23] and developed a series of

Raw Log

```
081109 205149 751 INFO dfs.DataNode$BlockReceiver: Changing
block file offset of block blk_4241467193520768333 from 0 to
30867456 meta file offset to 241159
```

**Log Parser****Structured Log**

Time	081109 205149 751
Level	INFO
System	dfs.DataNode\$BlockReceiver
Template	Changing block file offset of block (.*?) from ([-]?[0-9]+) to ([-]?[0-9]+) meta file offset to ([-]?[0-9]+)
Parameters	["blk_4241467193520768333", 0, 30867456, 241159]

Fig. 1. An example of log parsing on the HDFS dataset.

log parsing tools [24], [25], [26], [27]. In this work, we use the Drain parser [25] to process raw logs and generate log templates in order to provide structured data for our Transformer-based anomaly detector.

B. Log Anomaly Detection

In this work, we focus on the log anomaly detection task, which aims to analyze log data and recognize anomalous patterns automatically. The general process of log anomaly detection is shown in Fig. 2. First, raw logs are processed by log parsers to extract structured data, according to which log events are extracted. Then, the log events are sorted by timestamps, and thus time-series event sequences are acquired. After that, the extracted event sequences are input into the log anomaly detector. Finally, the detector identifies whether anomalous patterns exist in the event sequences. Therefore, the core technology of log anomaly detection is to learn and identify the key features that distinguish normal logs from anomalous ones [33].

Previous works have proposed numerous methods for log anomaly detection, which can be generally categorized into two types: (I) classic ML-based and (II) DL-based techniques. We systematically summarize and discuss them in the following.

1) *Classic ML-Based Methods*: Some works propose to extract statistical features and then train classic ML models for anomaly detection [14], [15], [28]. Xu et al. [14] propose to extract statistical features of events and parameters in raw logs. Then, they use the PCA algorithm to detect anomalies. Lin et al. [15] propose the LogCluster method, which clusters log sequences according to quantitative features and generates representative samples for anomaly detection. Restricted to the quality of extracted features and learning capabilities, these classic ML-based methods perform poorly in production environments. To alleviate this problem, Luo et al. [29] propose a Noise-Contrastive Learning (NCL) technique to create synthetic negative samples for binary classification and further use an ensemble of decision tree models to capture the dependencies

in log event sequences. However, these ML-based methods are still limited by feature engineering and difficulties in processing high-dimensional, contextual information in log data.

2) *DL-Based Methods*: To alleviate the shortcomings of classic ML-based methods, some works use DL models to learn sequential log features automatically [33]. Some research works preprocess raw logs into image-like data and propose to use convolutional neural networks (CNNs) to learn the spatial features end-to-end. Lu et al. [34] propose a framework combining the logkey2vec encoding technique and multiple 1D-CNN models to accurately detect anomalies on the HDFS dataset. Wang et al. [35] propose to use the temporal convolutional network (TCN) for log anomaly detection to leverage both temporal and spatial information in log sequences.

As system logs are essentially time-series semi-structured data, most DL-based methods detect abnormal patterns in a way similar to processing natural language. Some researchers propose to train recurrent neural network (RNN)-based models for anomaly detection [12], [13], [36], [37]. DeepLog [12] first suggests using the Long Short-Term Memory (LSTM) model [38] to learn the contextual features of normal log event sequences and detect abnormal patterns based on its predictions. LogAnomaly [13] extends DeepLog and proposes the Template2Vec approach to extract both sequential and quantitative features. Thanks to the richer information extracted by Template2Vec, LogAnomaly achieves stronger detection capabilities. Unlike these works, some researchers also use the Bi-directional LSTM (Bi-LSTM) models to learn the bi-directional contextual log features, which are reported to outperform traditional LSTM-based methods [39], [40]. Yu et al. [39] propose to use the attention-based Bi-LSTM model [41] to replace the classic LSTM model for prediction-based log anomaly detection. Zhang et al. [42] propose to train a sentiment language model based on logging-related source codes and use a Bi-LSTM model for log anomaly detection. In addition to the variants of LSTMs, researchers use the gated recurrent unit (GRU) model [43], which is also a commonly used RNN model, to learn sequential features in logs in order to achieve higher computational efficiency [44], [45], [46], [47].

Researchers also propose to use the autoencoder architecture for log anomaly detection. The autoencoder is a self-supervised learning model consisting of an encoder and a decoder, where the encoder compresses the input data into low-dimensional vectors, and the decoder aims to reconstruct them into the original data [48]. Autoencoder is originally designed for feature compression and generative tasks, but has recently been used for anomaly detection [49]. Zhang et al. [50] propose to use an attention-based deep neural network (DNN) to capture the contextual information in log data and then train an autoencoder-based architecture to detect anomalies based on the reconstruction loss. Wang et al. [51] propose to use the BERT model to acquire contextual features and use the variational autoencoder (VAE) [52] for log anomaly detection. Wadekar et al. [53] combines both convolutional autoencoder (CAE) and VAE models for hybrid, self-supervised log anomaly detection. Wu et al. [54] propose the Pre-LogMGAE framework

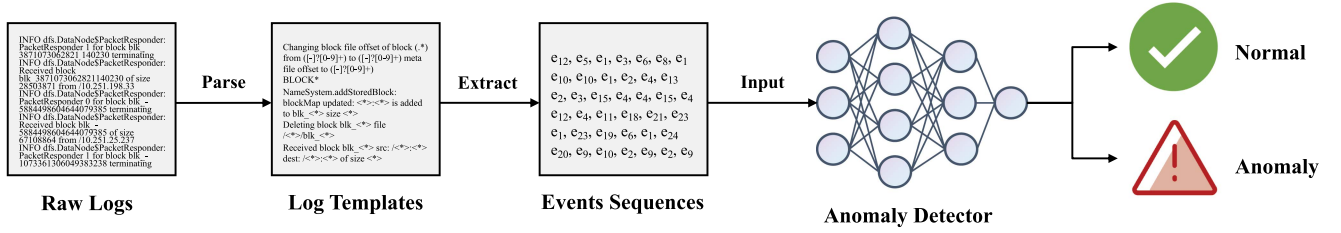


Fig. 2. The general process of log anomaly detection.

that combines the masked graph autoencoder model with contrastive learning for self-supervised pre-training and anomaly detection. In addition to the autoencoder, as a DL-based generative model, the generative adversarial network (GAN) is also introduced to the log anomaly detection task [55], [56].

With the rapid progress of DL, the Transformer architecture has taken enormous advantage in various fields [57], [58] and thus is leveraged for log anomaly detection [59], [60], [61]. Recently, researchers have also proposed to leverage the powerful learning capabilities of LLMs for log analysis [62], [63], [64]. Sakong et al. [65] propose a Transformer-based framework that uses the Drain parser and Doc2Vec embedding for reconstruction-based log anomaly detection. Guo et al. [30] propose the LogBERT framework, which trains a BERT-like model as the anomaly detector by generating masked tokens in normal sequences. Xiao et al. [31] propose Loader, which uses multiple Transformer layers to analyze log patterns and proposes a ‘Top-P’ algorithm to detect anomalies. In [1], we propose a detection framework to train a Transformer encoder-based detector with pre-training and fine-tuning approaches. Le et al. [32] propose to leverage the BERT model to process and analyze raw logs, reducing the reliance on log parsers. Then, the BERT model is adapted for a binary classification task. Recently, language models have also been applied for log anomaly detection. Guo et al. [66] propose a pre-training and fine-tuning pipeline to adapt language models for binary classification of log lines. Han et al. [17] propose to pre-train a GPT-2 language model and fine-tune it using reinforcement learning to detect anomalies. Although achieving high detection performance, these advanced DL-enhanced methods lead to relatively heavy runtime overhead. Ott et al. [67] propose to use different pre-trained language models, such as BERT and GPT-2, to generate embedded representations for log messages, to enhance the robustness and generalization capabilities of DL-based log anomaly detectors.

C. Generative Pre-Trained Transformers (GPTs)

Since being first proposed in 2017, the Transformer architecture has achieved dominant advantages in many fields, such as natural language processing [68], computer vision [69], recommender systems [70], etc. The original Transformer architecture contains multiple encoder and decoder layers, which leverage the positional encoding and self-attention mechanism to learn the complex contextual features in the input data [68]. As the scale of Transformer-based models continues to increase in these years, traditional supervised learning approaches are less

effective. In addition, the lack of labeled data is another difficulty in supervised learning. To this end, the ‘pre-training and fine-tuning’ paradigm is widely applied in the deep learning domain. For instance, researchers from OpenAI pre-train a Transformer decoder-based foundation language model with the next-token-prediction task, and then fine-tune it to downstream generative tasks with Reinforcement Learning (RL)-based algorithms to align with humans and acquire the Generative Pre-trained Transformers (GPTs) model [18], [71], [72].

The ‘pre-training and fine-tuning’ paradigm includes pre-training and fine-tuning phases. The goal of the pre-training phase is to learn the basic semantic features in input data, without considering the downstream tasks. Researchers typically use self-supervised learning to pre-train the foundation models. In the fine-tuning phase, the foundation models are further trained to fit specific downstream tasks. To fine-tune the foundation models, the parameters of their low-level layers are usually frozen, and they are optimized using labeled data from specific tasks. Our framework applies the pre-training and fine-tuning paradigm. We pre-train the foundation model to learn the general semantic features in log sequences. Then, we fine-tune the foundation model to the event prediction task. As introduced above, labeled log data is difficult to acquire in real-world scenarios. The pre-training phase is suitable for extracting and learning features in unlabeled data, which makes our framework perform well in such scenarios.

D. Knowledge Distillation

Rapid development of ML, especially DL technologies, has brought significant changes to many fields, but it also faces the challenges of high computing and storage overhead. For example, it is difficult to deploy large-scale DL models on embedded devices with limited energy and hardware resources. Therefore, DL-based models need to be compressed to be suitable for deployment in such scenarios. To this end, the knowledge distillation method is proposed to effectively compress the scale of DL models while retaining acceptable performance. As a transfer learning method, knowledge distillation leverages the ‘Teacher-Student’ architecture and aims to transfer dark knowledge, i.e., the hidden knowledge learned from training data, from a large-scale teacher model to a lightweight student model [73]. Hinton et al. [74] propose a temperature-based ($Temp$) Softmax function to extract dark knowledge and generate soft labels (q_i) from output logits (z_i), which is defined as

$$q_i = \frac{\exp(z_i / Temp)}{\sum_j \exp(z_j / Temp)}. \quad (1)$$

TABLE I
COMPARISON OF LOG ANOMALY DETECTION METHODS

Method	Class	Technical Highlights	Limitations
PCA [14]	ML	extract statistical features in events and parameters from raw logs, PCA algorithm for anomaly detection	limited by feature engineering
LogCluster [15]	ML	extract statistical features, generate representative samples, clustering algorithm for anomaly detection	limited by feature engineering
iForest [28]	ML	ensemble of decision trees, linear inference complexity	limited representation abilities
STrees [29]	ML	noise-contrastive learning method, an ensemble of decision tree models, binary classification	limited representation abilities
DeepLog [12]	DL	LSTM event sequence modeling, prediction-based detection	limited sequential information
LogAnomaly [13]	DL	Template2Vec encoding algorithm, LSTM-based model, prediction-based anomaly detection	limited learning abilities
LogBERT [30]	DL	pre-training BERT language model, masked language modeling, prediction-based anomaly detection	reliance on labeled data
Loader [31]	DL	multi-layer Transformer-based model for log analysis, Top-P anomaly detection algorithm	reliance on labeled data
NeuralLog [32]	DL	BERT language model for log analysis, a template-free method, binary classification	ineffectiveness on unknown anomalies
LogGPT [33]	DL	GPT-2 large language model for log analysis, RL-based fine-tuning and anomaly detection	heavy training and inference overhead
Our Method	DL	multi-layer Transformer, the pre-training and fine-tuning paradigm, multi-level knowledge distillation technique	additional training cost

The knowledge distillation uses both soft labels and ground truth labels (i.e., hard labels) to train the lightweight student model. Unlike hard labels, soft labels contain the understanding and preferences of the teacher model, thus improving the generalization capabilities of the trained student model [74]. In recent years, numerous knowledge distillation methods have been proposed to adapt to different model compression tasks, such as adversarial distillation, multi-teacher distillation, cross-modal distillation, etc [75], [76], [77]. Due to the efficient transfer learning abilities of knowledge distillation, it is also applied in natural language processing, computer vision, recommender systems, and cybersecurity domains [73], [78], [79].

In this work, we leverage the powerful model compression and knowledge-transfer capabilities of the knowledge distillation techniques and propose a multi-level distillation method suitable for log data.

E. Summary

We systematically review and compare the representative log anomaly detection methods in Table I. Although existing approaches have achieved strong detection performance, several critical challenges remain to be addressed:

- As system logs grow increasingly complex and large-scale, there is a need to develop more powerful log anomaly detection models.
- In practice, high-quality, manually labeled log data for training is scarce, yet most existing methods rely

heavily on such data. Therefore, the information contained in unlabeled logs should be fully exploited.

- Most existing studies focus primarily on performance while overlooking runtime overhead, which limits their applicability in real-world deployments.

In this work, we propose a hybrid log anomaly detection framework to address the aforementioned challenges. The framework adopts a pre-training and fine-tuning paradigm to fully exploit unlabeled logs and reduce the reliance on labeled training data. It employs a multi-layer Transformer encoder architecture to effectively capture contextual dependencies in log event sequences. Furthermore, to enhance its practicality in real-world deployments, the framework integrates a multi-level knowledge distillation technique to achieve accurate and cost-efficient anomaly detection.

III. OVERVIEW OF OUR FRAMEWORK

In this paper, we introduce the pre-training and fine-tuning paradigm to the log analysis domain and propose a hybrid log anomaly detection framework. We first pre-train a foundation model on large-scale unlabeled log data to learn the basic sequential features. Then, we fine-tune it on labeled logs to discern outlier patterns and gain anomaly detection capabilities. After that, the fine-tuned model is further compressed while preserving its performance. The trained detector identifies anomalous patterns by comparing its predictions with ground-truth events.

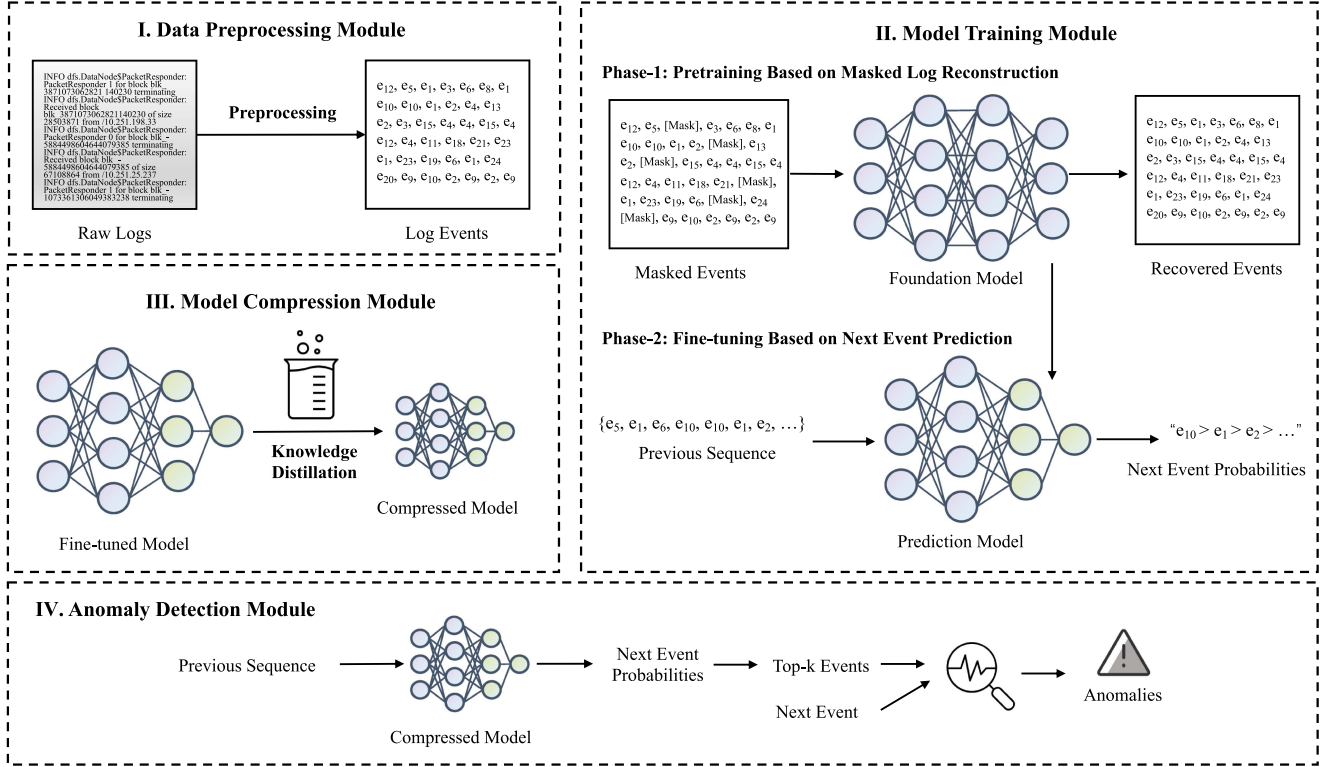


Fig. 3. The overview of our framework.

Our framework exploits sequential information within logs to achieve better performance in real-world scenarios.

As shown in Fig. 3, our framework includes four modules: (i) Data Preprocessing, (ii) Model Training, (iii) Model Compression, and (iv) Anomaly Detection. Specifically, raw logs are processed into event sequences in the first module. The second module uses the extracted sequences to train the foundation model and then the prediction model with self-supervised learning. The third module uses the knowledge distillation technique to compress the fine-tuned model. Finally, the compressed prediction model serves as the anomaly detector in the last module. We give a detailed introduction to our framework as follows.

A. Data Preprocessing Module

Loggers typically populate predefined templates and print out logs, which are formatted strings that record critical runtime information of computer systems. Therefore, massive raw logs can be processed into events according to log templates, which significantly reduces the amount of input data. We sort the extracted events by timestamp to generate structured event sequences. After that, these event sequences serve as input data for subsequent modules.

B. Model Training Module

According to previous works [12], [13], there are specific sequential patterns within logs that distinguish normal sequences from abnormal ones. The objective of log anomaly detection is to identify these patterns from time series log data, which is

very similar to processing and analyzing natural language. In other words, our aim is to understand the language that the log data speaks. Therefore, we design our model training module referring to some state-of-the-art techniques in the NLP field.

Log event sequences are first encoded into vectors. Since the scale of log templates is relatively small in common systems, we select One-Hot encoding as the vectorization approach, which is simple yet sufficient for our task. Our model training module follows the pre-training and fine-tuning paradigm. Specifically, in the pre-training phase, we propose the masked log reconstruction-based technique to train a foundation model, which learns the contextual features in event sequences. Then, in the fine-tuning phase, we train the foundation model to make correct predictions on the next log events in normal sequences, which actually gives the model the ability to learn normal patterns. After that, the fine-tuned prediction model is further compressed in the next module.

C. Model Compression Module

Existing research works rarely consider the computational efficiency or storage overhead of log anomaly detectors. Thus, in the model compression module, we aim to leverage knowledge distillation to train a compressed model while preserving the high performance of the fine-tuned prediction model. Specifically, we propose a multi-level knowledge distillation method suitable for sequential log data. Our knowledge distillation method calculates three separate loss values to train the compressed model so that the compressed model achieves high

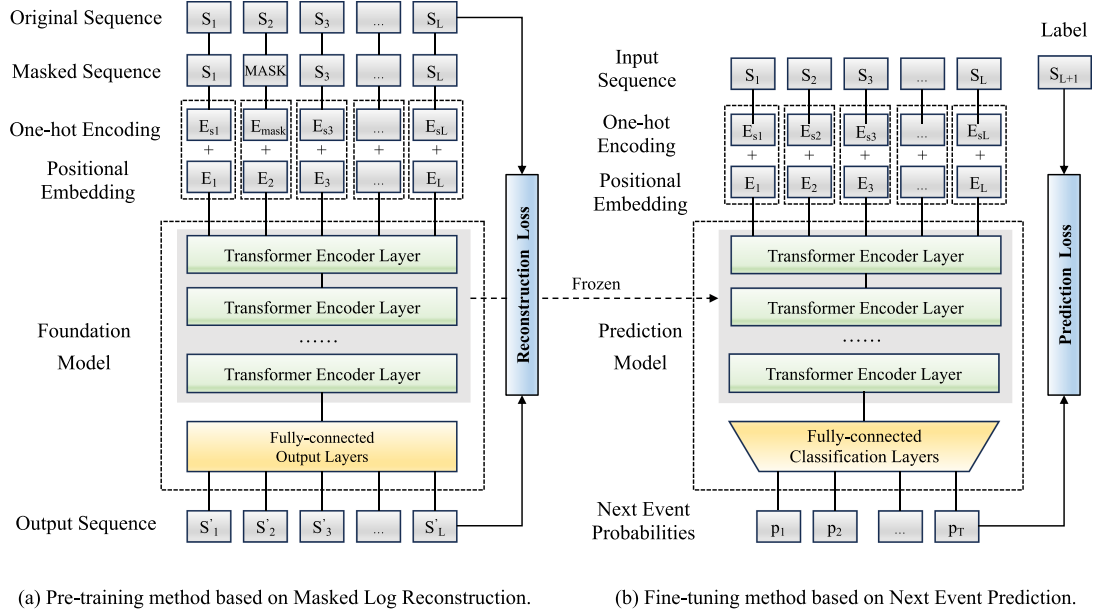


Fig. 4. An illustration of our proposed pre-training and fine-tuning methods. After pre-training, the Transformer layers (green part) are frozen and serve as the backbone of the fine-tuned model.

prediction performance while accurately fitting the fine-tuned model. After model compression, the lightweight distilled model serves as the anomaly detector in the next module.

D. Anomaly Detection Module

We deploy the compressed event prediction model as the log anomaly detector, which continuously predicts the next events based on previous event sequences. In particular, the fine-tuned model from the second module can also serve as an anomaly detector because the model compression module only reduces its scale. Since the prediction model is trained to learn normal patterns, it detects anomalies simply by comparing the predictions with actual events.

IV. METHODOLOGY

In this section, we give a detailed introduction to our pre-training, fine-tuning, and knowledge distillation methods. These methods are leveraged in the second and third modules of our log anomaly detection framework.

A. Pre-Training Based on Masked Log Reconstruction

In the pre-training phase, the foundation model learns the semantic features. Transformers [68] have achieved dominant advantages in multiple fields, which are extremely suitable for processing time series data and significantly outperform RNN models. Therefore, we use the Transformer as the basis of our foundation model. It leverages multiple Transformer encoder layers to learn the sequential patterns and concatenates fully connected layers to generate outputs.

Inspired by BERT [80] and Masked Autoencoder [81], we pre-train the foundation model using input masking techniques,

the core idea of which is to train models to learn key features by recovering masked inputs. In this paper, we introduce the input masking technique to the field of log analysis and propose a masked log reconstruction approach for pre-training. Our approach randomly masks part of the input events and trains the model to automatically recover the original sequences. The process of our mask-based pre-training methods is shown in Fig. 4(a).

To illustrate our approach, suppose that an original sequence S_{origin} has L events, then we have $S_{origin} = [S_{origin}^1, S_{origin}^2, S_{origin}^3, \dots, S_{origin}^L]$. To acquire the masked sequence, we randomly choose $\lfloor L * r \rfloor$ events with a masking rate (r) and replace them with a masking token ($[mask]$). Assuming that the second event is masked, the masked sequence (S_{mask}) is formulated as $S_{mask} = [S_{origin}^1, [mask], S_{origin}^3, \dots, S_{origin}^L]$. Given masked sequences, the foundation model is trained to accurately recover the masked sequences, i.e., to maximize the probabilities of outputting the corresponding original sequences. Therefore, the loss function is defined as the distance between the output vector ($\mathbf{X}_{recover}$) and the original vector ($\mathbf{X}_{origin}$), where $\mathbf{X}_{origin}$ is the vectorized input sequence. In this paper, we leverage the Mean-Squared Error (MSE) loss to measure the distance. Supposing that $\mathbf{X}^j$ is the j^{th} value in the vectors, the loss function of our pre-training approach is formulated as

$$\begin{aligned} Loss_{pt} &= MSE_Loss(\mathbf{X}_{origin}, \mathbf{X}_{recover}) \\ &= \frac{1}{L} \sum_{j=1}^L \left(\mathbf{X}_{origin}^j - \mathbf{X}_{recover}^j \right)^2. \end{aligned} \quad (2)$$

To train the foundation model, we continuously update its parameters to minimize the loss value. After pre-training, the

foundation model obtains the contextual information in the input log sequences. The trained foundation model is further fine-tuned in the next phase.

B. Fine-Tuning Based on Next Event Prediction

As mentioned above, the foundation model learns the semantic features from the log data. Although the pre-trained model contains valuable information, it is insufficient for detecting anomalies. Therefore, we fine-tune the foundation model to a downstream task, i.e., next event prediction, which aims to predict the next events according to previous sequences. We train the model on labeled data so that it learns specific patterns that distinguish normal sequences from abnormal ones.

To accomplish the prediction task, as shown in Fig. 3, we initially modify the structure of the trained foundation model by removing its output layers and freezing the parameters of the Transformer encoder layers since the semantic features are contained in these layers. Then, we concatenate a fully connected classifier to the modified foundation model, which aims to output the probabilities of the next events. Actually, other ML models that have prediction capabilities can act as the classifier, e.g., LSTM, 1D-CNN, etc. We fine-tune the prediction model by continuously optimizing the parameters of the concatenated classifier. Consequently, the fine-tuning phase requires much less training time than pre-training.

As shown in Fig. 4(b), the modified model takes previous sequences as inputs and outputs the predictions of the next events, which are vectors containing the probabilities that different events occur next. To train the prediction model, we iteratively sample fixed-length sequences from labeled datasets as inputs and the next events as labels. Given previous sequences, the model aims to maximize the probabilities that target events are correctly predicted. Therefore, the loss function is defined as the difference between the output probabilities ($\mathbf{Y}_{output}$) and the vectorized target label ($\mathbf{Y}_{true}$). Using the Cross-Entropy (CE) loss to measure the difference, the loss function is formulated as

$$\begin{aligned} Loss_{ft} &= CE_Loss(\mathbf{Y}_{output}, \mathbf{Y}_{true}) \\ &= - \sum_{i=1}^n \mathbf{Y}_{true}^i \log \mathbf{Y}_{output}^i, \end{aligned} \quad (3)$$

where n is the number of log event types and $\mathbf{Y}^i$ is the probability value of the i^{th} event.

In the fine-tuning phase, all the training data are extracted from normal logs. After training, the prediction model learns normal sequential patterns and can accurately predict normal events. The fine-tuned prediction model is adopted for log anomaly detection.

C. Knowledge Distillation for Efficient Log Anomaly Detection

The storage and computing efficiency of log anomaly detectors is critical to deployment in real-world scenarios. However, existing research works rarely consider either storage or computing efficiency issues, which leads to difficulties in real-world

deployment. To this end, we introduce the knowledge distillation technique into our framework. In this work, we propose a multi-level knowledge distillation method suitable for log data, which are essentially large-scale serialized sequences of discrete log events. We distill our fine-tuned prediction model using our knowledge distillation method to compress it. The distilled prediction model has a significant decrease in the number of parameters, storage usage, and inference time. We will give a detailed introduction to our proposed knowledge distillation method in the following.

As introduced in Section II, knowledge distillation is a widely applied compression method in the DL domain. In the classic knowledge distillation framework, the student model continuously learns knowledge from the teacher model, so that the small-scale student model can achieve generalization capabilities similar to the teacher model. In our framework, the fine-tuned prediction model serves as the teacher, while a small-scale transformer-based model is trained as the student. The student model has a similar structure to the teacher model, with fewer Transformer encoder layers and a smaller hidden vector size.

The overview of our knowledge distillation method is shown in Fig. 5. To train the student model, we first need to extract fixed-length input sequences from the training dataset. Let X_{input} be a vectorized input sequence that is fed into both the teacher and student models. The teacher model then produces a logit $\mathbf{Y}_{teacher}$ and the student model produces a logit $\mathbf{Y}_{student}$. After that, these output logits are further calculated by the Softmax function to acquire the output probabilities, i.e., $\mathbf{Q}_{teacher}$ and $\mathbf{Q}_{student}$. Suppose that $\mathbf{Y}^j$ is the probability of the j^{th} value in $\mathbf{Y}$, $\mathbf{Q}^i$ is the probability of the i^{th} event. The Softmax function is calculated by comparing a single logit with others, which is formulated as

$$\mathbf{Q}^i = \frac{\exp(\mathbf{Y}^i / Temp)}{\sum_{j=1}^n \exp(\mathbf{Y}^j / Temp)}, \quad (4)$$

where $Temp$ is the temperature parameter. A larger temperature value makes the output probability distribution smoother. The output probability vector of the teacher model ($\mathbf{Q}_{teacher}$) is the soft label of the input sequence. Since the goal of knowledge distillation is to compress the teacher model while preserving its generalization capabilities, the student model is trained to fit the teacher model. In other words, given an input sequence, the student model is trained to minimize the distance between the soft label ($\mathbf{Q}_{teacher}$) and the output vector ($\mathbf{Q}_{student}$). Using the KL Divergence method to measure the distance, the distillation loss function is defined as

$$\begin{aligned} Loss_{distill} &= D_{KL}(\mathbf{Q}_{student} || \mathbf{Q}_{teacher}) \\ &= \sum_{i=1}^n \mathbf{Q}_{student}^i \log \frac{\mathbf{Q}_{student}^i}{\mathbf{Q}_{teacher}^i}, \end{aligned} \quad (5)$$

where i is the index of $\mathbf{Q}_{teacher}$ and $\mathbf{Q}_{student}$.

In addition to training the student model to fit the teacher model, we also expect the student model to perform well on the log event prediction task. To this end, the student model is trained to fit the ground-truth labels, not just soft labels. Therefore, the training goal is similar to the fine-tuning phase, which is

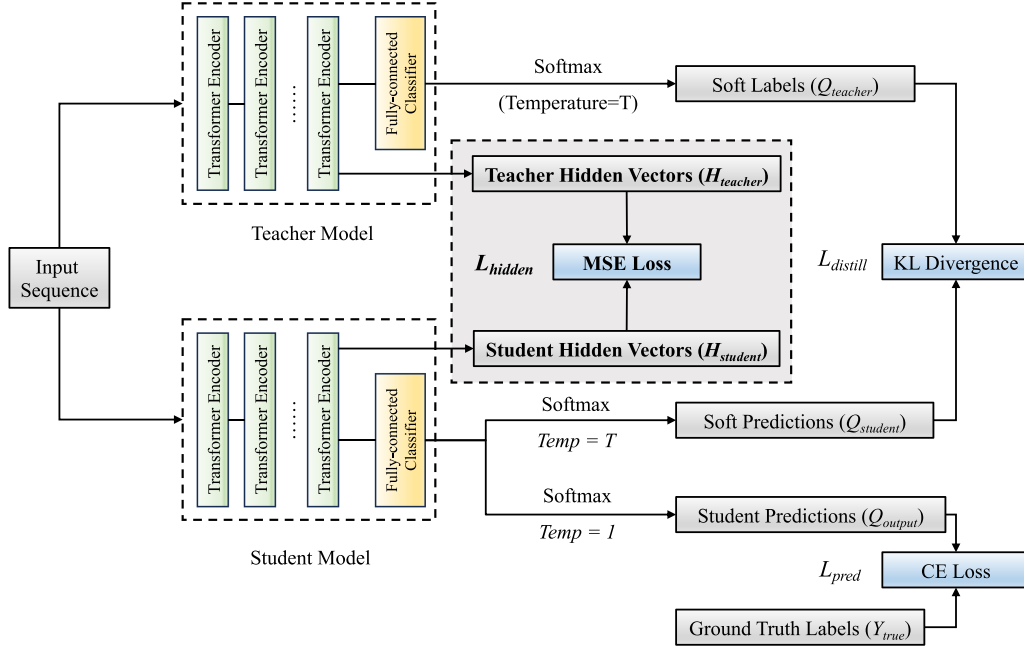


Fig. 5. The overview of our knowledge distillation method. As in the shaded area, we extend the conventional knowledge distillation method by introducing an additional loss term to measure the distance between the hidden representations of two models.

to maximize the probability that the student model accurately predicts the ground truth event. The loss function is defined as the distance between the output vector ($Q_{student}$) and the target label (Y_{true}). Using CE loss, the prediction loss function is formulated as

$$\begin{aligned} Loss_{pred} &= CE_Loss(Q_{output}, Y_{true}) \\ &= - \sum_{i=1}^n Y_{true}^i \log Q_{output}^i. \end{aligned} \quad (6)$$

Unlike conventional knowledge distillation methods, we propose to learn not only the outputs of the teacher model but also its hidden representations in intermediate layers. The Transformer layers in our prediction model include the sequential features learned from log event sequences. In order to make the student model learn more knowledge from the teacher model, we propose to extend the conventional knowledge distillation method by additionally training the student model to fit the outputs of the hidden layers in the teacher model. As shown in the shaded area in Fig. 5, we extract the output of the final Transformer encoder layer from both the student and teacher models, i.e., $H_{student}$ and $H_{teacher}$. Our training goal is to maximize the probability that the student model outputs the same hidden vector as the teacher model. Therefore, the loss function is defined as the distance between $H_{student}$ and $H_{teacher}$. Using MSE loss to measure the distance, the hidden loss function is formulated as

$$\begin{aligned} Loss_{hidden} &= MSE_Loss(H_{student}, H_{teacher}) \\ &= \frac{1}{h} \sum_{j=1}^h (H_{student}^j - H_{teacher}^j)^2, \end{aligned} \quad (7)$$

assuming that h is the size of the hidden vector and j is the index of the hidden vector.

The overall loss function of our knowledge distillation method is defined as the combination of the above three loss functions:

$$Loss_{KD} = \alpha \cdot Loss_{distill} + \beta \cdot Loss_{pred} + \gamma \cdot Loss_{hidden}, \quad (8)$$

where α , β , and γ are parameters to control the ratio of these losses. To train the student model, we iteratively extract fixed-length log event sequences from the training dataset as input. Then, we feed the input sequences into both the teacher and student models to acquire outputs and calculate the loss values, based on which the parameters of the student model are updated. After knowledge distillation, the student model acquires comparable performance with the original prediction model with much lower computational and storage overhead.

D. Anomaly Detection With Pre-Trained Transformers

After the pre-training, fine-tuning, and knowledge distillation phases, the prediction model effectively learns the sequential patterns of normal logs. It is capable of predicting the next events in normal logs according to previous sequences. Therefore, the behaviors of a functioning system can be precisely predicted by the model. In this way, outlier system operations that are beyond the expectations of the model are detected as anomalies. Inspired by this, we implement a prediction-based anomaly detection approach in our framework. Specifically, given input sequences, the trained prediction model outputs vectors containing the probabilities that certain log events will occur next. As the fine-tuned prediction model makes accurate predictions on normal logs, the ground truth events are expected to appear in the events with top- k probabilities; otherwise, anomaly patterns

Algorithm 1: Log Anomaly Detection.

Input : S, k and $window_size$
Output: Res_List

```

1 Initialize  $Res\_List$  to store detection results;
2 for  $i \leftarrow 1$  to  $len(S)$  do
3    $seq \leftarrow S[i]$ ;
4   for  $j \leftarrow 1$  to  $len(seq) - window\_size$  do
5     // extract input and label
6      $input \leftarrow seq[j : j + window\_size]$ ;
7      $next\_event \leftarrow seq[j + window\_size]$ ;
8      $output \leftarrow M(input)$ ;
9      $pred \leftarrow P(output, k)$ ;
10    if  $next\_event$  is not in  $pred$  then
11      // an anomaly is detected
12       $res \leftarrow Abnormal$ ;
13      break;
14    if  $(j + window\_size) == len(seq)$  then
15      // end of the sequence
16       $res \leftarrow Normal$ ;
17  append  $res$  to  $Res\_List$ ;
18 return  $Res\_List$ 

```

are identified. The algorithm for online anomaly detection is presented in Algorithm 1.

V. EVALUATION SETTINGS

A. Datasets

We evaluate our method on three popular public log datasets [82], including the HDFS dataset [83], the BGL dataset [84], and the Thunderbird [84] dataset. The detailed information about these datasets is introduced as follows.

HDFS: The Hadoop File System (HDFS) dataset contains 11,175,629 logs collected from a Hadoop cluster deployed on 200 Amazon EC2 nodes. These logs record 29 different types of log events in the Hadoop system. The dataset is further processed and labeled by human experts. There are 575,061 blocks in the cluster, 16,838 of which are manually labeled as abnormal.

BGL: The BGL dataset is collected from the BlueGene/L (BGL) supercomputer system at Lawrence Livermore National Labs (LLNL). The BGL dataset contains 4,747,963 logs, among which 348,460 logs are labeled abnormal. These logs include 377 different types of log events in the BGL supercomputer.

Thunderbird (TBird): The TBird dataset records runtime logs collected from the Thunderbird supercomputer system at Sandia National Labs (SNL). As in previous work, we extract the first 20,000,000 lines of log messages from the raw data, among which 41,543 lines are labeled as abnormal. After log parsing, these logs include 1,207 different types of log events in the Thunderbird supercomputer.

B. Baselines

In this paper, we compare our framework with nine baseline methods. For ML-based methods, we evaluate PCA [83],

Isolation Forest [28], and LogCluster [15] implemented in Loglizer.¹ For DL-based methods, we evaluate six state-of-the-art anomaly detectors, including Deep Autoencoder, DeepLog [12], LogAnomaly [13], Loader [31], LogGPT [17], and LogBERT [30].

C. Implementation Details

For the HDFS dataset, we use the regex tool to extract log events according to the open-source log templates provided by Xu et al. [83]. For BGL and TBird datasets, we use the Drain parser to extract log templates and structured log data from raw logs [24], [25]. We use the sliding window to extract fixed-length sequences with $sequence_length = 10$ and $step_size = 1$. Then, we leverage one-hot encoding (HDFS and BGL) and word embedding (TBird) to obtain vectorized representations. In the pre-training phase, we initialize the Transformer-based foundation model with $hidden_size = 128$. We set the number of Transformer encoder layers as 12 for the HDFS dataset and 4 for the BGL dataset. We randomly mask 10% of the input events to pre-train the foundation model. To detect anomalies, considering the size of event templates, the value of k is set as 3, 39, and 120 for the HDFS, BGL, and TBird datasets, respectively.

For RNN-based baselines, we set $num_layers = 2$ and $hidden_size = 64$ for both the DeepLog and LogAnomaly models. For Transformer-based baselines, we set $hidden_size = 64$, $num_layers = 4$, and $num_headers = 4$ for the Deep Autoencoder, Loader, and LogBERT. We train the above DL-based baseline models for 500 epochs to achieve the best performance. For the LLM-enhanced LogGPT method, we set $num_layers = 4$ and $num_headers = 4$ for the GPT-2 language model. We pre-train LogGPT for 100 epochs and fine-tune it for 20 epochs with the RL algorithm to achieve the best performance.

To evaluate our knowledge distillation method, we set $num_layers = 1$ and $hidden_size = 8$ for the student model. The fine-tuned prediction model serves as the teacher model. We set $\alpha = 1.0$, $\beta = 1.0$, and $\gamma = 2.0$ for our knowledge distillation method. For the baseline distillation method, which only leverages distillation and prediction losses, we set $\alpha = 1.0$ and $\beta = 1.0$. We train both student models for 100 epochs to achieve the best performance.

We run all the experiments on a server with Ubuntu 20.04, Intel Xeon 2.90 GHz CPU, 256 GB memory, and NVIDIA RTX A6000 GPUs. We implement our method and the baseline models with Python 3.9 and PyTorch 2.5.

D. Evaluation Metrics

In our experiments, we count the TP, FP, TN, and FN values and compute the following three metrics to evaluate all methods, including $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$, and $F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall}$.

To evaluate our knowledge distillation method, we calculate three indicators to measure the computational and storage

¹Loglizer. <https://github.com/logpai/loglizer>. [9]

TABLE II
EVALUATION OF OVERALL PERFORMANCE

Dataset Method	Precision	HDFS Recall	F1 Score	Precision	BGL Recall	F1 Score	Precision	TBird Recall	F1 Score
PCA	15.33%	100.00%	26.58%	50.30%	99.80%	66.89%	77.74%	100.00%	87.47%
iForest	95.98%	88.97%	92.34%	100.00%	20.36%	33.83%	93.62%	0.18%	0.35%
LogCluster	100.00%	63.64%	77.78%	100.00%	36.44%	53.41%	99.98%	48.25%	65.09%
Autoencoder	96.84%	87.73%	92.06%	20.69 %	58.01%	30.52%	85.34%	99.98%	92.08%
DeepLog	97.62%	96.55%	97.08%	99.03%	96.68%	97.84%	99.77%	99.96%	99.87%
LogAnomaly	96.87%	98.74%	97.79%	97.61%	96.79%	97.20%	99.77%	99.97%	99.87%
Loader	97.65%	94.40%	96.26%	75.54%	99.41%	99.05%	99.84%	99.96%	99.91%
LogGPT	93.92%	91.76 %	92.15%	98.66%	98.66%	98.66%	96.95%	96.84%	96.68%
LogBERT	99.03%	98.95%	98.99%	99.06%	95.74%	97.37%	99.81%	99.28%	99.54%
Our Method	99.50%	99.43%	99.47%	99.28%	98.81%	99.05%	99.95%	99.96%	99.96%

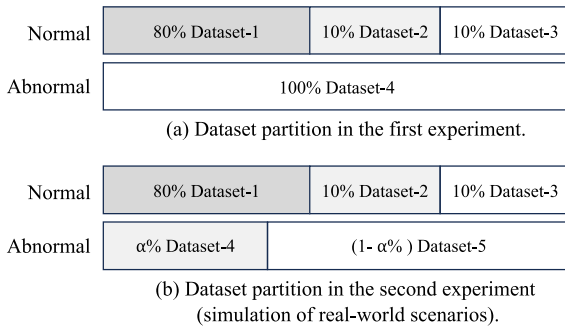


Fig. 6. Dataset partition in our experiments.

overhead of log anomaly detectors, including the number of parameters, the number of Floating Point Operations (FLOPs), and inference time (Infer Time) for each input sequence. For the inference time metric, we first warm up the GPUs for 1,000 repetitions and then measure the inference time of a single input sequence (*batch_size*=1) 1,000 times. We report the average inference time to ensure reliable results.

VI. EVALUATION RESULTS AND ANALYSIS

In this section, we aim to answer the following research questions to evaluate our proposed framework:

RQ1: How is the overall detection performance of the proposed framework compared to the SOTA baselines?

RQ2: How does the proposed framework perform in real-world scenarios where high-quality labeled training data is scarce?

RQ3: How effective is the proposed knowledge distillation approach? How much is the inference overhead of the compressed model for real-world deployment?

RQ4: Is the proposed knowledge distillation approach robust to real-world scenarios?

A. Evaluation of Overall Performance (*RQ1*)

In the first experiment, we compare the overall performance of our framework with baselines on the HDFS, BGL, and TBird datasets. It is notable that the HDFS dataset is divided according

to *block_id*, and the BGL and TBird datasets are divided by time. These datasets are split as shown in Fig. 6(a). For our framework, we pre-train the foundation model on Dataset 1 and fine-tune it on Dataset 2. For the baselines, we train them on a dataset combining Dataset 1 and Dataset 2. We test all the methods on Dataset 3 and Dataset 4.

The experimental results are shown in Table II. Overall, our framework achieves the best performance among all methods, which indicates that it learns semantic features from normal log data and effectively detects outlier log patterns. Although ML-based methods achieve high performance in specific metrics (e.g., PCA achieves 100% recall on the HDFS and TBird datasets), their overall performance is much lower than DL-based methods, proving that such methods have lower learning abilities than DL models.

Among all the DL-based methods, AE achieves relatively poor results (92.06% F1 score on HDFS, 30.52% F1 score on BGL, and 92.08% F1 score on TBird). Especially on the BGL dataset, AE has serious overfitting problems. The other six DL-based models, including DeepLog, LogAnomaly, Loader, LogGPT, LogBERT, and our framework, achieve higher detection performance. On the HDFS dataset, our method achieves 99.50% precision, 99.43% recall, and 99.47% F1 score, all of which outperform the DL-based baseline models. On the BGL dataset, the DL-based baselines achieve comparable performance while our method achieves 99.28% precision, 98.81% recall, and 99.05% F1 score, significantly outperforming other DL-based models. Besides, on the TBird dataset, all the DL-based methods perform well, and our method achieves the best performance among them with 99.96% F1 score. These experimental results indicate that our method detects more anomalies while triggering fewer false alarms, which is critical for deploying anomaly detectors in production environments.

It is also shown that the Transformer-based architecture and the training methods in our framework play a significant role. Among the DL-based detectors, our method, Loader, and LogBERT utilize the Transformer Encoder-based architecture to extract and learn the sequential features within log data, while DeepLog and LogAnomaly use the LSTM model. The evaluation results show that these Transformer-based methods outperform the LSTM-based ones on the HDFS and BGL

datasets, demonstrating the superiority of the Transformer-based architecture. Moreover, our method performs better than Loader and LogBERT models on all three datasets, which indicates that, following the pre-training and fine-tuning paradigm, our training method makes the model learn more key features and better distinguish unexpected patterns.

To conclude, in the first experiment, we train and evaluate our method and nine baseline models on three public datasets, where all the data are assumed to be labeled, which conducts an overall evaluation of all these methods and provides unified metrics of their performance. Experimental results show that our framework achieves the best performance among these methods. In the following experiment, we will further prove the superiority of our framework in real-world scenarios.

B. Evaluation of Real-World Scenarios (RQ2)

In the second experiment, we compare the performance of our framework and the DL-based baselines in real-world scenarios. As described above, existing works typically train their anomaly detection models on normal log data and test them on both normal and abnormal data. They assume that all the log data in the dataset is labeled. However, labeled log datasets are extremely difficult to obtain, as manually labeling logs requires a heavy workload. Therefore, the amount of unlabeled log data is significantly larger than the labeled data in real-world scenarios. We believe that it is essential to evaluate anomaly detectors in such circumstances.

We reconstruct the HDFS dataset to simulate real-world scenarios. As shown in Fig. 6(b), abnormal data is divided into two parts, where Dataset 4 contains $\alpha\%$ of it and Dataset 5 contains the rest. For our framework, we first pre-train the foundation model on an unlabeled dataset combining Dataset 1 and Dataset 4, and then fine-tune it on Dataset 2. We train the baselines on a dataset combining Dataset 1, Dataset 2, and Dataset 4 to obtain fair evaluation results. All methods are tested on Dataset 3 and Dataset 5.

We select the value of α as 20, 40, and 60 to construct our datasets and run the experiments on these datasets separately. As shown in Table III, the evaluation metrics prove that our framework remarkably outperforms the baselines. More importantly, the performance of the baseline models drops significantly compared to the first experiment ($\alpha = 0$), while our framework achieves similar performance, making our method the only one that can adapt to real-world scenarios where unlabeled logs are leveraged for training. As trained on unlabeled datasets, the baselines learn the sequential features within normal and abnormal logs, making these patterns more indistinct and thus resulting in significant degradation in performance. Unlike them, our framework first pre-trains the foundation model to exploit unlabeled data and then fine-tunes it on labeled data to acquire detection capabilities. Consequently, our framework performs well in real-world scenarios.

In addition, it is also shown that the performance of our framework decreases slightly as α increases. With more abnormal data combined in the pre-training phase, the foundation model learns more abnormal patterns. Given the same fine-tuning dataset, it

TABLE III
EVALUATION OF REAL-WORLD SCENARIOS ON THE HDFS DATASET

Ratio	Methods	Precision	Recall	F1 Score
20%	DeepLog	95.82%	67.92%	79.49%
	LogAnomaly	86.20%	45.39%	59.46%
	Loader	92.17%	31.31%	46.74%
	LogGPT	88.19%	86.86%	87.33%
	LogBERT	95.43%	64.19%	76.75%
	Our Method	99.35%	99.44%	99.39%
40%	DeepLog	93.74%	62.58%	75.05%
	LogAnomaly	82.16%	46.78%	59.63%
	Loader	89.32%	32.55%	47.71%
	LogGPT	90.87%	88.36%	89.15%
	LogBERT	93.25%	53.95%	68.35%
	Our Method	99.20%	99.43%	99.31%
60%	DeepLog	90.85%	59.56%	71.95%
	LogAnomaly	70.87%	29.14%	41.30%
	Loader	84.05%	24.57%	38.02%
	LogGPT	88.42%	85.69%	86.81%
	LogBERT	90.27%	56.95%	69.84%
	Our Method	98.74%	95.12%	96.89%

is more difficult for the fine-tuned model to distinguish between normal and abnormal patterns. Despite a slight degradation in detection performance, our framework is the only method that achieves acceptable evaluation results.

To summarize, in the second experiment, we prove that our framework is suitable for real-world scenarios. Among all DL methods, our framework is the only one that achieves high performance in such scenarios.

C. Evaluation of Knowledge Distillation (RQ3)

In the third experiment, we evaluate the performance of our model compression method on HDFS, BGL, and TBird datasets. As introduced above, the knowledge distillation framework consists of a teacher model and a student model, aiming to transfer knowledge from the teacher to the student. In our framework, the fine-tuned model serves as the teacher model, while a Transformer-based model with a similar architecture serves as the student model. It is worth noting that the student model has fewer layers and smaller hidden sizes than the teacher model, as our goal is to compress the fine-tuned model. We evaluate our knowledge distillation method on the datasets of the first experiment. In addition to the performance metrics, we also calculate the number of parameters, FLOPs, and inference time for the teacher model, the student model, and DL-based baselines, in order to prove the advantages of our knowledge distillation method.

The evaluation results are shown in Table IV. On all three datasets, the student model achieves comparative performance to the teacher model. On HDFS and TBird datasets, the distilled model slightly overfits on normal data, thus detecting a few more false positives. Despite this, the distilled student model achieves high detection performance on all three datasets (99.56% on HDFS, 99.05% on BGL, and 99.93% on TBird), proving that our

TABLE IV
EVALUATION OF OUR KNOWLEDGE DISTILLATION METHOD

Dataset	Method	Precision	Recall	F1 Score	# Parameters	FLOPs	Infer Time
HDFS	DeepLog	97.62%	96.55%	97.08%	52.51K	0.51M	0.41ms
	LogAnomaly	96.87%	98.74%	97.79%	104.99K	1.03M	0.74ms
	Loader	97.65%	94.40%	96.26%	221.02K	1.13M	4.71ms
	LogGPT	93.92%	91.76%	92.15%	213.84K	12.83M	6.47ms
	LogBERT	99.03%	98.95%	98.99%	138.14K	0.68M	0.74ms
	Teacher Model	99.50%	99.43%	99.47%	221.02K	1.13M	5.01 ms
	Student Model	99.36%	99.75%	99.56%	11.02K	0.02M	0.56 ms
BGL	DeepLog	99.03%	96.68%	97.84%	76.43K	0.54M	0.41ms
	LogAnomaly	97.61%	96.79%	97.20%	152.46K	1.08M	0.76ms
	Loader	75.54%	99.41%	99.05%	2.02M	5.84M	1.58ms
	LogGPT	98.66%	98.66%	98.66%	217.62K	13.06M	5.33ms
	LogBERT	99.06%	95.74%	97.37%	18.56K	0.70M	0.71ms
	Teacher Model	99.28%	98.81%	99.05%	2.02M	5.84M	1.79 ms
	Student Model	99.28%	98.81%	99.05%	1.61M	1.71M	0.57 ms
TBird	DeepLog	99.77%	99.96%	99.87%	128.95K	0.59M	1.03ms
	LogAnomaly	99.77%	99.97%	99.87%	256.79K	1.18M	1.75ms
	Loader	99.84%	99.96%	99.91%	1.68M	2.90M	1.55ms
	LogGPT	96.95%	96.84%	96.68%	939.65K	56.38M	6.76ms
	LogBERT	99.81%	99.28%	99.54%	145.98K	0.76M	1.09ms
	Teacher Model	99.95%	99.96%	99.96%	1.83M	2.63M	0.84ms
	Student Model	99.90%	99.97%	99.93%	1.58M	1.89M	0.36ms

knowledge distillation method is capable of retaining the high detection performance of original detectors. More importantly, the compressed model has low computational overhead and storage usage. On the HDFS dataset, our knowledge distillation method reduces the number of parameters by 95.01%, the number of FLOPs by 98.23%, and the inference time by 88.82%. On the BGL dataset, our method reduces the number of parameters by 20.29%, the number of FLOPs by 70.72%, and the inference time by 68.16%. Meanwhile, on the TBird dataset, our method reduces the number of parameters by 13.76%, the number of FLOPs by 28.14%, and the inference time by 57.14%. The significant decline in these indicators demonstrates that our knowledge distillation method effectively improves the computational and storage efficiency of the fine-tuned prediction model. Moreover, it is also shown in Table IV that the distilled student model achieves higher detection performance than the DL-based baselines with much lower computational overhead.

To conclude, the third experiment proves that our knowledge distillation method can significantly compress the scale of log anomaly detectors while preserving comparable detection performance. After knowledge distillation, the lightweight model has significantly lower runtime overhead, which is suitable for real-world deployment where computational and storage resources are limited.

D. Evaluation of Our Knowledge Distillation Approach in Real-World Scenarios (RQ4)

In this experiment, we evaluate the robustness of our knowledge distillation approach in real-world scenarios where labeled data is scarce. To this end, we use the same dataset as the second experiment. Specifically, we leverage the proposed knowledge

TABLE V
EVALUATION OF KNOWLEDGE DISTILLATION IN REAL-WORLD SCENARIOS

Ratio	Model	Precision	Recall	F1 Score
20%	Teacher	99.35%	99.44%	99.39%
	Student	99.28%	99.48%	99.38%
40%	Teacher	99.20%	99.43%	99.31%
	Student	97.51%	99.97%	98.72 %
60%	Teacher	98.74%	95.12%	96.89%
	Student	94.12%	99.29%	96.63%

distillation method to compress the three models trained in the second experiment, which are pre-trained with 20%, 40%, and 60% abnormal data, respectively.

The evaluation results are shown in Table V. It is shown that the detection performance of all three student models decreases slightly after knowledge distillation, which is caused by the reduction in capacity of the lightweight models. Despite the slight performance degradation, our distilled lightweight model achieves significantly higher performance than the DL-based baselines, as shown in Table III. In addition, the advantages are even greater when more abnormal data are mixed. These results prove that our distilled lightweight models perform well in the simulated real-world scenarios, demonstrating the robustness of our knowledge distillation approach.

E. Ablation Study on Training Pipeline

To illustrate the effectiveness of our hybrid training method, we compare it with a Transformer-based prediction model trained with supervised learning. The baseline shares the same

TABLE VI
ABLATION STUDY ON TRAINING PIPELINE

Dataset	Method	Precision	Recall	F1 Score
HDFS	Supervised	97.65%	94.90%	96.26%
	Hybrid (Ours)	99.50%	99.43%	99.47%
BGL	Supervised	75.54%	99.41%	85.85%
	Hybrid (Ours)	99.28%	98.81%	99.05%
TBird	Supervised	99.84%	99.96%	99.91%
	Hybrid (Ours)	99.95%	99.96%	99.96%

TABLE VII
COMPARATIVE EVALUATION OF PRE-TRAINING TASKS

Dataset	Method	Precision	Recall	F1 Score
HDFS	MLM + FT	96.66%	90.32%	93.38%
	MLR + FT (ours)	99.50%	99.43%	99.47%
BGL	MLM + FT	98.55%	96.91%	97.73%
	MLR + FT (ours)	99.28%	98.81%	99.05%
TBird	MLM + FT	99.82%	99.97%	99.89%
	MLR + FT (ours)	99.95%	99.96%	99.96%

architecture as our fine-tuned model and is trained with a supervised task, i.e., log event prediction.

We conduct experiments on a dataset as that in the first experiment. The results are shown in Table VI. Overall, our training method performs better than the supervised learning method on all datasets. Since these models share the same architecture, the evaluation results demonstrate the advancement of our hybrid training pipeline.

F. Evaluation of Pre-Training Task

To evaluate the effectiveness of our proposed masking-based pre-training task, i.e., Masked Log Reconstruction (MLR), we conduct a comparative experiment against the BERT MLM approach. In the experiments, we pre-train two foundation models with the MLR and MLM tasks, respectively, and then fine-tune them on the next event prediction task for anomaly detection. The model architecture, the fine-tuning approach, and all hyperparameters are set to the same for the two methods. The evaluation results are shown in Table VII. It is shown that, pre-trained with our proposed MLR task, the detection performance of fine-tuned detectors is significantly improved on all three datasets, demonstrating the advantage of our proposed pre-training approach tailored to log anomaly detection.

G. Ablation Study on Knowledge Distillation

We evaluate our Multi-Level Knowledge Distillation (MLKD) approach in this experiment. We compare it to the conventional knowledge distillation framework, which only uses the prediction and distillation losses to transfer knowledge. We evaluate these two methods on the HDFS, BGL, and TBird datasets.

TABLE VIII
ABLATION STUDY ON KNOWLEDGE DISTILLATION

Dataset	Method	Precision	Recall	F1 Score
HDFS	w/o MLKD	99.38%	99.56%	99.47%
	w/ MLKD (ours)	99.36%	99.75%	99.56%
BGL	w/o MLKD	99.28%	98.81%	99.05%
	w/ MLKD (ours)	99.28%	98.81%	99.05%
TBird	w/o MLKD	99.84%	98.97%	99.90%
	w/ MLKD (ours)	99.90%	99.97%	99.93%

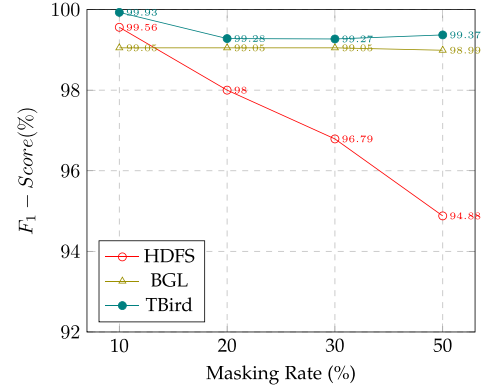


Fig. 7. Evaluation of masking rate in pre-training.

The evaluation results are shown in Table VIII. On the HDFS and TBird datasets, our MLKD method achieves higher F1 Scores than conventional knowledge distillation. On the BGL dataset, limited by the data scale, our method achieves comparative performance to the baseline. These results indicate that, trained with the additional hidden loss, the student model acquires more knowledge from the teacher model. In conclusion, we prove that our multi-level knowledge distillation method exceeds conventional methods in knowledge transfer capabilities.

H. Evaluation of Hyperparameter Sensitivity

1) *Masking Rate*: We evaluate the effectiveness of our masking-based pre-training approach under diverse input masking rates, including 10%, 20%, 30%, and 50%, on three public datasets. The evaluation results are presented in Fig. 7. It is shown that the detection performance decreases with bigger masking rates, and the best performance is achieved with masking rate = 10%.

2) *Temperature Coefficient*: We evaluate the effectiveness of our knowledge distillation method with diverse temperature settings, including 0.1, 0.5, 1.0, 2.0, and 5.0. The evaluation results are presented in Fig. 9. It is shown from the figure that the optimal knowledge distillation results occur when the temperature coefficient is set to 1.0 or 2.0.

3) *Multi-Level Knowledge Distillation Coefficients*: We evaluate the contributions of the three coefficients, i.e., α , β , and γ , in our multi-level knowledge distillation approach (8). We conduct a grid search over the three coefficients, each taking values from {1.0, 2.0}, and then evaluate the distilled detectors

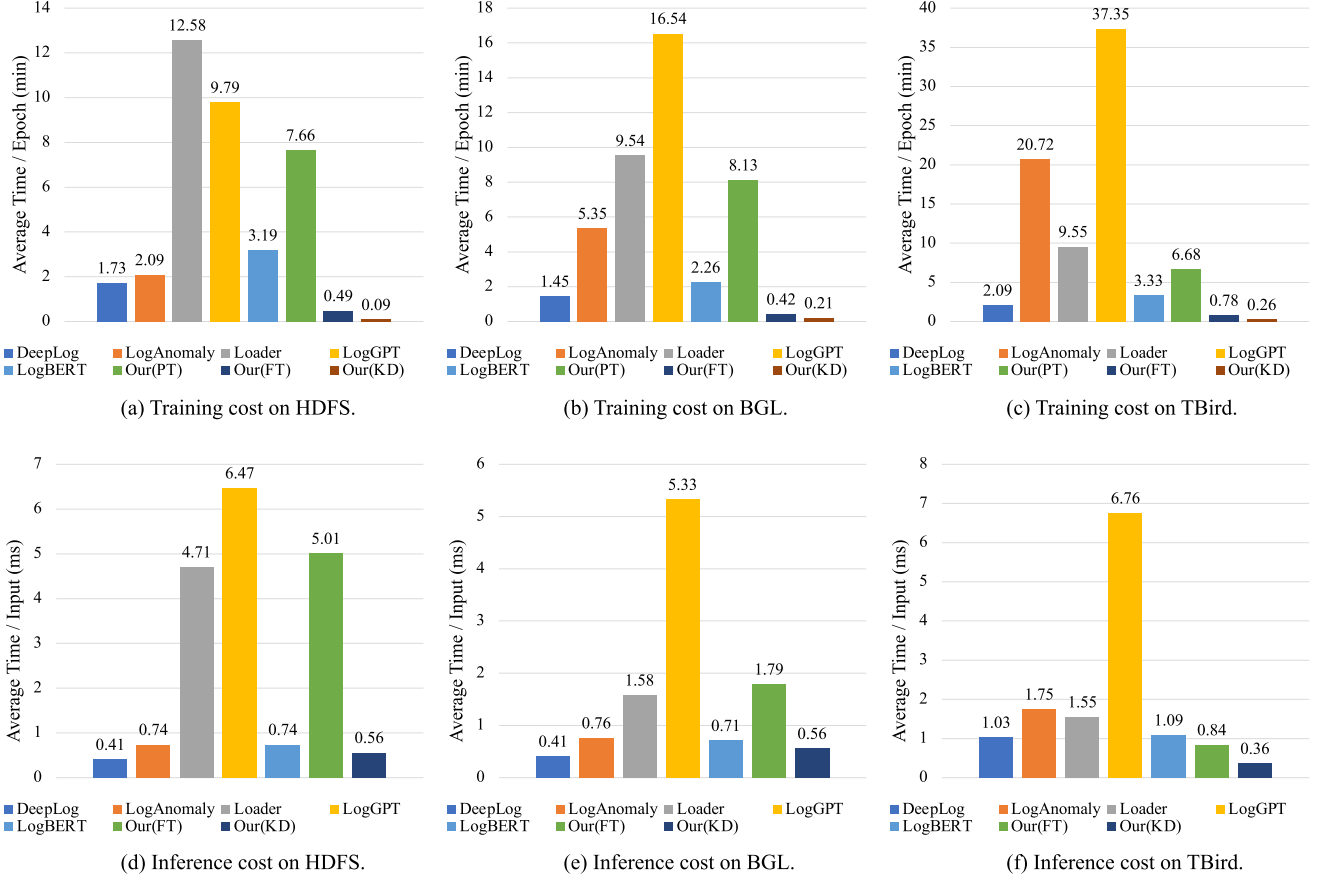


Fig. 8. Training and inference time consumption of DL-based models.

TABLE IX
EVALUATION OF MULTI-LEVEL KNOWLEDGE DISTILLATION COEFFICIENTS

Coefficient Setting (α - β - γ)	HDFS F1 Score	BGL F1 Score	TBird F1 Score
1.0-1.0-1.0	99.48%	99.05%	99.75%
1.0-1.0-2.0 (ours)	99.56%	99.05%	99.93%
1.0-2.0-1.0	99.53%	89.43%	99.76%
1.0-2.0-2.0	99.48%	99.05%	99.76%
2.0-1.0-1.0	99.51%	89.37%	99.75%
2.0-1.0-2.0	99.34%	99.05%	99.74%
2.0-2.0-1.0	99.55%	99.05%	99.75%
2.0-2.0-2.0	99.50%	89.37%	99.75%

accordingly. The evaluation results are presented in Table IX. The coefficient setting reported in our work ($\alpha=1.0$, $\beta=1.0$, $\gamma=2.0$) achieves the best performance across datasets.

I. Evaluation of Time Consumption

The application of DL technology has greatly improved the performance of log anomaly detectors, but their runtime overhead has also increased. The time consumption of the training and inference phases in DL-based approaches is critical to real-world deployment. Specifically, the inference time affects the feasibility of actual deployment, while the training time

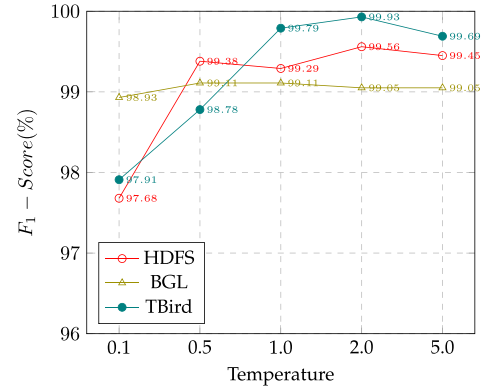


Fig. 9. Evaluation of temperature settings in knowledge distillation.

affects the overhead of online training of the detector. To this end, we measure the training and inference time cost of our method and the DL-based baselines, which are shown in Fig. 8. Different phases of model training, including pre-training (PT), fine-tuning (FT), and knowledge distillation (KD), are evaluated separately.

For inference overhead, it is shown that the inference time of our fine-tuned model is relatively low among all DL-based methods. After knowledge distillation, the inference cost is further reduced. With the high detection performance, our framework is

extremely suitable for real-world deployment. For training time consumption, the training overhead of our framework is higher than the RNN-based baseline (DeepLog and LogAnomaly), but is comparable to the Transformer-based methods. We believe that the additional training consumption of our framework is acceptable, considering its outstanding performance.

J. Limitations and Future Work

Despite the high performance and efficiency, our study has potential limitations. First, our data preprocessing and input encoding approaches are based on pre-defined log templates. Although log templates can be automatically extracted by parsers, such template-based methods hardly generalize to new types of logs. To this end, we suggest studying LLM-enhanced template-free methods that directly process unstructured, natural language-like log data. Second, our Transformer encoder-based framework introduces additional runtime overhead compared to conventional ML and DL-based methods. Therefore, in future work, we aim to investigate the application of other lightweight techniques, such as model quantization and pruning, in this field. Finally, due to the robustness issues of DL models, it is possible that our approach is vulnerable to AI-based attacks, such as adversarial examples and backdoor attacks, motivating us to study the adversarial robustness of DL-based detectors in future work.

VII. CONCLUSION

In this work, we have proposed a hybrid framework for log anomaly detection. Our framework follows the pre-training and fine-tuning paradigm to leverage log data and achieve better learning capabilities. We have proposed the masked log reconstruction approach to pre-train a Transformer-based foundation model and fine-tune it to detect anomalous log patterns. We have also proposed a knowledge distillation technique to compress the fine-tuned model while maintaining its performance. Evaluation results on three widely used public datasets show that our framework outperforms existing state-of-the-art methods and is ideal for real-world applications.

REFERENCES

- [1] S. Yan et al., "Log-based anomaly detection with transformers pre-trained on large-scale unlabeled data," in *Proc. IEEE Int. Conf. Commun.*, Denver, CO, USA, Jun. 2024, pp. 1–6.
- [2] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *ACM Comput. Surv.*, vol. 57, no. 6, pp. 1–39, Feb. 2025.
- [3] B. Yuan et al., "Minimizing financial cost of DDoS attack defense in clouds with fine-grained resource management," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2541–2554, Oct.–Dec. 2020.
- [4] Y. Otoum, A. Asad, and A. Nayak, "Blockchain meets adaptive honeypots: A trust-aware approach to next-gen IoT security," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 2270–2287, 2026.
- [5] Y. Gong, C. Zhang, and Y. Liu, "Zero-day malware defence with limited samples," *J. Commun. Inf. Netw.*, vol. 9, no. 4, pp. 340–347, 2024.
- [6] Y. Qin, X. Yang, L.-X. Yang, and K. Huang, "Mitigating social engineering attacks through cost-effective security awareness training policy," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 3145–3158, Jul./Aug. 2025.
- [7] Y. Xia, X. Liu, Y. Zhao, and Y. Wang, "FCLA-DT: Federated continual learning with authentication for distributed digital twin-based industrial IoT," *J. Commun. Inf. Netw.*, vol. 9, no. 4, pp. 362–373, 2024.
- [8] A. J. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," *Commun. ACM*, vol. 55, no. 2, pp. 55–61, 2012.
- [9] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience Report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Softw. Rel. Eng.*, Ottawa, ON, Canada, Oct. 2016, pp. 207–218.
- [10] M. Du and F. Li, "Spell: Online streaming parsing of large unstructured system logs," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 11, pp. 2213–2227, Nov. 2019.
- [11] R. Christensen and F. Li, "Adaptive log compression for massive log data," in *Proc. SIGMOD Conf.*, Jun. 2013, pp. 1283–1284.
- [12] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dallas, TX, USA, Oct. 2017, pp. 1285–1298.
- [13] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. Int. Joint Conf. Artif. Intell.*, Macao, China, Aug. 2019, pp. 4739–4745.
- [14] W. Xu, L. Huang, A. Fox, D. A. Patterson, and M. I. Jordan, "Online system problem detection by mining patterns of console logs," in *Proc. IEEE Int. Conf. Data Mining*, Miami, FL, USA, Dec. 2009, pp. 588–597.
- [15] Q. Lin, H. Zhang, J. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. IEEE 38th Int. Conf. Softw. Eng. Companion*, May 2016, pp. 102–111.
- [16] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Halifax, NS, Canada, Aug. 2017, pp. 665–674.
- [17] X. Han, S. Yuan, and M. Trabelsi, "LogGPT: Log anomaly detection via GPT," in *Proc. IEEE BigData*, Sorrento, Italy, Dec. 2023, pp. 1117–1122.
- [18] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Neural Inf. Process. Syst.*, Nov. 2022, pp. 27730–27744.
- [19] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 130:1–130:37, 2022.
- [20] Z. Chen, J. Liu, W. Gu, Y. Su, and M. R. Lyu, "Experience report: Deep learning-based system log analysis for anomaly detection," 2021, *arXiv: 2107.05908*.
- [21] V. Beck, M. Landauer, M. Wurzenberger, F. Skopik, and A. Rauber, "System log parsing with large language models: A review," 2025, *arXiv:2504.04877*.
- [22] Y. Liu et al., "Interpretable online log analysis using large language models with prompt strategies," in *Proc. IEEE/ACM Int. Conf. Program. Comprehension*, Lisbon, Portugal, Apr. 2024, pp. 35–46.
- [23] Z. Ma, D. J. Kim, and T. P. Chen, "LibreLog: Accurate and efficient unsupervised log parsing using open-source large language models," in *Proc. IEEE/ACM ICSE*, Ottawa, ON, Canada, May 2025, pp. 924–936.
- [24] J. Zhu et al., "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng., Softw. Eng. Pract.*, Montreal, QC, Canada, May 2019, pp. 121–130.
- [25] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Serv.*, Honolulu, HI, USA, Jun. 2017, pp. 33–40.
- [26] M. Du and F. Li, "Spell: Streaming parsing of system event logs," in *Proc. IEEE 16th Int. Conf. Data Mining*, Barcelona, Spain, Dec. 2016, pp. 859–864.
- [27] H. Hamooni, B. Debnath, J. Xu, H. Zhang, G. Jiang, and A. Mueen, "LogMine: Fast pattern recognition for log analytics," in *Proc. 25th ACM Int. Conf. Inform. Knowl. Manage.*, Indianapolis, IN, USA, Oct. 2016, pp. 1573–1582.
- [28] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining*, Pisa, Italy, Dec. 2008, pp. 413–422.
- [29] P. Luo et al., "System log anomaly detection with noise-contrastive learning and pattern feature," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 11–24, 2026.
- [30] H. Guo, S. Yuan, and X. Wu, "LogBert: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Netw.*, Shenzhen, China, Jul. 2021, pp. 1–8.
- [31] T. Xiao et al., "Loader: A log anomaly detector based on transformer," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3479–3492, Sep./Oct. 2023.
- [32] V. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Melbourne, VIC, Australia, Nov. 2021, pp. 492–504.
- [33] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Mach. Learn. Appl.*, vol. 12, 2023, Art. no. 100470.

- [34] S. Lu, X. Wei, Y. Li, and L. Wang, "Detecting anomaly in Big Data system logs using convolutional neural network," in *Proc. IEEE 16th Int. Conf. Dependable, Autonomic Secure Comput., 16th Intl Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol. Congr.*, Aug. 2018, pp. 151–158.
- [35] Z. Wang, J. Tian, H. Fang, L. Chen, and J. Qin, "LightLog: A lightweight temporal convolutional network for log anomaly detection on the edge," *Comput. Netw.*, vol. 203, 2022, Art. no. 108616.
- [36] R. Xiao, H. Chen, J. Lu, W. Li, and S. Jin, "AllInfoLog: Robust diverse anomalies detection based on all log features," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 2529–2543, Sep. 2023.
- [37] Y. Fadili, Y. El Yamani, J. Kilani, N. El Kamoun, Y. Baddi, and F. Bensalah, "An enhancing timeseries anomaly detection using LSTM and Bi-LSTM architectures," in *Proc. 11th Int. Conf. Wireless Netw. Mobile Commun.*, 2024, pp. 1–6.
- [38] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [39] D. Yu, X. Hou, C. Li, Q. Lv, Y. Wang, and N. Li, "Anomaly detection in unstructured logs using attention-based Bi-LSTM network," in *Proc. IEEE Int. Conf. Netw. Intell. Digit. Content*, Beijing, China, Nov. 2021, pp. 403–407.
- [40] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust and unified deep learning based log anomaly detection for diverse faults," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, Coimbra, Portugal, Oct. 2020, pp. 92–103.
- [41] Z. Huang, W. Xu, and K. Yu, "Bidirectional LSTM-CRF models for sequence tagging," 2015, *arXiv: 1508.01991*.
- [42] D. Zhang, D. Dai, R. Han, and M. Zheng, "SentiLog: Anomaly detecting on parallel file systems via log-based sentiment analysis," in *Proc. ACM HotStorage. Virtual Event*, Jul. 2021, pp. 86–93.
- [43] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv: 1412.3555*.
- [44] A. Farzad, "Log message anomaly detection with oversampling," *Int. J. Artif. Intell. Appl.*, vol. 11, no. 4, pp. 53–65, 2020.
- [45] S. Gu, Y. Chu, W. Zhang, P. Liu, Q. Yin, and Q. Li, "Research on system log anomaly detection combining two-way slice GRU and GA-attention mechanism," in *Proc. 4th Int. Conf. Artif. Intell. Big Data*, 2021, pp. 577–583.
- [46] C. Li, J. Mo, G. Zhou, and G. Li, "Log anomaly detection based on parallel fusion of CNN and GRU," in *Proc. Int. Joint Conf. Neural Netw.*, 2024, pp. 1–8.
- [47] W. Yuan, S. Ying, X. Duan, H. Cheng, Y. Zhao, and J. Shang, "MidLog: An automated log anomaly detection method based on multi-head GRU," *J. Syst. Softw.*, vol. 226, 2025, Art. no. 112431.
- [48] P. Li, Y. Pei, and J. Li, "A comprehensive survey on design and application of autoencoder in deep learning," *Appl. Soft Comput.*, vol. 138, 2023, Art. no. 110176.
- [49] K. Berahmand, F. Daneshfar, E. S. Salehi, Y. Li, and Y. Xu, "Autoencoders and their applications in machine learning: A survey," *Artif. Intell. Rev.*, vol. 57, no. 2, 2024, Art. no. 28.
- [50] L. Zhang et al., "LogAttn: Unsupervised log anomaly detection with an autoencoder based attention mechanism," in *Proc. Int. Conf. Knowl. Sci., Eng. Manage.*, Tokyo, Japan, Aug. 2021, vol. 12817, pp. 222–235.
- [51] Q. Wang, X. Zhang, X. Wang, and Z. Cao, "Log sequence anomaly detection method based on contrastive adversarial training and dual feature extraction," *Entropy*, vol. 24, no. 1, 2022, Art. no. 69.
- [52] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Representations*, Banff, AB, Canada, Apr. 14–16, 2014, pp. 1–14.
- [53] A. Wadekar, T. Gupta, R. Vijan, and F. Kazi, "Hybrid CAE-VAE for unsupervised anomaly detection in log file systems," in *Proc. 10th Int. Conf. Comput., Commun. Netw. Technol.*, Kanpur, India, Jul. 2019, pp. 1–7.
- [54] A. Wu and Y. Kwon, "Pre-LogMGAE: Identification of log anomalies using a pre-trained masked graph autoencoder," in *Proc. 43rd Int. Symp. Reliable Distrib. Syst.*, Charlotte, NC, USA, Sep. 2024, pp. 294–306.
- [55] X. Han and S. Yuan, "Unsupervised cross-system log anomaly detection via domain adaptation," in *Proc. 30th ACM Int. Conf. Inform. Knowl. Manage.*, Queensland, Australia, Nov. 2021, pp. 3068–3072.
- [56] X. Duan, S. Ying, W. Yuan, H. Cheng, and X. Yin, "A generative adversarial networks for log anomaly detection," *Comput. Syst. Sci. Eng.*, vol. 37, no. 1, pp. 135–148, 2021.
- [57] M. Ma, L. Han, and C. Zhou, "Research and application of transformer based anomaly detection model: A literature review," 2024, *arXiv:2402.08975*.
- [58] B. Liu, X. Liu, S. Gao, X. Cheng, and L. Yang, "LLM4CP: Adapting large language models for channel prediction," *J. Commun. Inf. Netw.*, vol. 9, no. 2, pp. 113–125, 2024.
- [59] S. Huang et al., "HitAnomaly: Hierarchical transformers for anomaly detection in system log," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2064–2076, Dec. 2020.
- [60] Q. Du, L. Zhao, J. Xu, Y. Han, and S. Zhang, "Log-Based anomaly detection with multi-head scaled dot-product attention mechanism," in *Proc. Int. Conf. Database Expert Syst. Appl.*, Virtual Event, Sep. 2021, vol. 12923, pp. 335–347.
- [61] R. Xiao, W. Li, J. Lu, and S. Jin, "ContextLog: Non-parsing log anomaly detection with all information preservation and enhanced contextual representation," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 4, pp. 4750–4762, Aug. 2024.
- [62] C. Almodovar, F. Sabrina, S. Karimi, and S. A. Azad, "LogFit: Log anomaly detection using fine-tuned language models," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1715–1723, Apr. 2024.
- [63] Z. Zhang, S. Li, L. Zhang, J. Ye, C. Hu, and L. Yan, "LLM-LADE: Large language model-based log anomaly detection with explanation," *Knowl. Based Syst.*, vol. 326, 2025, Art. no. 114064.
- [64] X. Li et al., "AnomalyGen: An automated semantic log sequence generation framework with LLM for anomaly detection," 2025, *arXiv: 2504.12250*.
- [65] W. Sakong, J. Kwon, K. Min, S. Wang, and W. Kim, "Anomaly transformer ensemble model for cloud data anomaly detection," *IEEE Trans. Cloud Comput.*, vol. 12, no. 4, pp. 1305–1313, Oct.–Dec. 2024.
- [66] H. Guo et al., "Logformer: A pre-train and tuning pipeline for log anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, Vancouver, BC, Canada, Feb. 2024, pp. 135–143.
- [67] H. Ott, J. Bogatinovski, A. Acker, S. Nedelkoski, and O. Kao, "Robust and transferable anomaly detection in log data using pre-trained language models," in *Proc. IEEE/ACM Int. Workshop Cloud Intell.*, 2021, pp. 19–24.
- [68] A. Vaswani et al., "Attention is all you need," in *Proc. Neural Inf. Process. Syst.*, Long Beach, CA, USA, Dec. 2017, pp. 5998–6008.
- [69] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, Virtual Event, Austria, May 2021, pp. 1–22.
- [70] F. Sun et al., "Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proc. 28th ACM Int. Conf. Inform. Knowl. Manage.*, Beijing, China, Nov. 2019, pp. 1441–1450.
- [71] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Proc. Neural Inf. Process. Syst.*, Long Beach, CA, USA, Dec. 2017, pp. 4299–4307.
- [72] OpenAI, "GPT-4 technical report," 2023, *arXiv: 2303.08774*.
- [73] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 2021.
- [74] G. E. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [75] X. Wang, R. Zhang, Y. Sun, and J. Qi, "KDGAN: Knowledge distillation with generative adversarial networks," in *Proc. Neural Inf. Process. Syst.*, Montréal, QC, Canada, Dec. 2018, pp. 783–794.
- [76] S. Gupta, J. Hoffman, and J. Malik, "Cross modal distillation for supervision transfer," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Las Vegas, NV, USA, Jun. 2016, pp. 2827–2836.
- [77] N. Papernot, M. Abadi, Ú. Erlingsson, I. J. Goodfellow, and K. Talwar, "Semi-supervised knowledge transfer for deep learning from private training data," in *Proc. Int. Conf. Learn. Representations*, Toulon, France, Apr. 2017, pp. 1–16.
- [78] D. Liu, P. Cheng, Z. Dong, X. He, W. Pan, and Z. Ming, "A general knowledge distillation framework for counterfactual recommendation via uniform data," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inform. Retrieval*, Virtual Event, China, Jul. 2020, pp. 831–840.
- [79] N. Papernot, P. D. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *Proc. IEEE Symp. Secur. Privacy*, San Jose, CA, USA, May 2016, pp. 582–597.
- [80] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186.

- [81] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. B. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, New Orleans, LA, USA, Jun. 2022, pp. 15979–15 988.
- [82] M. Landauer, F. Skopik, and M. Wurzenberger, "A critical review of common log data sets used for evaluation of sequence-based anomaly detection techniques," in *Proc. ACM Softw. Eng.*, 2024, vol. 1, pp. 1354–1375.
- [83] W. Xu, L. Huang, A. Fox, D. A. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. Int. Conf. Mach. Learn.*, Haifa, Israel, Jun. 2010, pp. 37–46.
- [84] A. J. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Edinburgh, U.K., Jun. 2007, pp. 575–584.



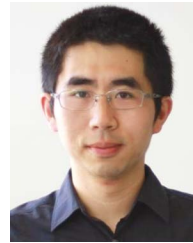
Senming Yan (Graduate Student Member, IEEE) received the B.E. degree in software engineering from Xidian University, Xi'an, China, in 2020. He is currently working toward the Ph.D. degree with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing. His research interests include network security, zero trust security, and AI for security.



Lei Shi (Graduate Student Member, IEEE) received the B.E. degree from Electronic Information School, Wuhan University, Wuhan, China, in 2021. She is currently working toward the Ph.D. degree with the School of Integrated Circuits (School of Electronic Information and Electrical Engineering), Shanghai Jiao Tong University, Shanghai, China. Her research interests include network security, intrusion detection, cybersecurity situational awareness, and AI for security.



Jing Ren (Member, IEEE) received the B.E. and Ph.D. degrees in communication engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2007 and 2015, respectively. She is currently an Associate Researcher with UESTC and Research Assistant with Peng Cheng Laboratory, Shenzhen, China. Her research interests include network architecture and protocol design, software-defined networking, and network security.



Wei Wang (Senior Member, IEEE) received the B.S. degree from Central South University, Changsha, China, in 2010, the M.S. degree from Southeast University, Nanjing, China, in 2013, and the Ph.D. degree from The University of New South Wales, Sydney, NSW, Australia, in 2017. From 2018 to 2021, he was a Postdoctoral Research Fellow with The University of New South Wales. He is currently a Senior Research Scientist (equivalent to Associate Professor) with Peng Cheng Laboratory, Shenzhen, China. His research interests include millimeterwave communications and machine learning for wireless communications. Dr. Wang was the recipient of 2023 IEEE ComSoc AP Outstanding Young Researcher Award and Best Paper Awards in the IEEE ICC 2016, 2024. He is the Editor of IEEE TRANSACTIONS ON MOBILE COMPUTING. He was the Editor of IEEE WIRELESS COMMUNICATIONS LETTERS and Co-Chair of multiple symposia and workshops at major IEEE conferences.



Limin Sun received the B.S. and Ph.D. degrees from the National University of Defense Technology, Changsha, China, in 1988 and 1998, respectively. He is currently a Professor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. He is also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing. His research interests include mobile vehicle networks, Internet of Things security, and wireless sensor networks.



Wei Zhang (Fellow, IEEE) received the Ph.D. degree from The Chinese University of Hong Kong, Hong Kong, in 2005. He is currently a Professor with the School of Electrical Engineering and Telecommunications, University of New South Wales, Sydney, NSW, Australia. His research focuses on 6 G communications and networks. He was the recipient of the IEEE Communications Society Joseph LoCicero Award for Exemplary Service to Publications in 2024. He was an IEEE ComSoc Distinguished Lecturer from 2016 to 2017. Within the IEEE ComSoc, he has taken many leadership positions, including the Chair of the Wireless Communications Technical Committee from 2019 to 2020, Vice Director of the Asia-Pacific Board from 2016 to 2021, Editor-in-Chief of IEEE WIRELESS COMMUNICATIONS LETTERS from 2016 to 2019, Member-at-Large on the Board of Governors from 2018 to 2020, Technical Program Committee Chair of APCC 2017 and ICC 2019 and 2024, Award Committee Chair of the Asia-Pacific Board, and Award Committee Chair of the Technical Committee on Cognitive Networks. He has been the Vice President of IEEE Communications Society, since 2022.